# 数据库管理系统原理与实现

杜小勇 陈红 卢卫 主编

深圳技术大学
大数据与互联网学院

# 关系数据库简介

- **提出关系模型的是美国IBM公司的E.F.Codd**
  - 1970年提出关系数据模型

    E.F.Codd, “A Relational Model of Data for Large Shared Data Banks”, 《Communication of the ACM》,1970
  - 之后，提出了关系代数和关系演算的概念
  - 1972年提出了关系的第一、第二、第三范式
  - 1974年提出了关系的BC范式

# 第二章 关系数据模型

- 2.1 关系模型的数据结构及形式化定义
- 2.2 关系代数
- 2.3 关系的完整性
- 2.4 小结

# 关系模型的数据结构及形式化定义

- 2.1 关系
- 2.2 关系模式
- 2.3 关系数据库
- 2.4 关系模型的存储结构

# 关系

- 单一的数据结构----关系

  现实世界的实体以及实体间的各种联系均用关系来表示

- 逻辑结构----二维表

  从用户角度，关系模型中数据的逻辑结构是一张二维表

- 建立在集合代数的基础上

# 关系

1. 域（Domain）
2. 笛卡尔积（Cartesian Product）
3. 关系（Relation）

# 关系

**域**是一组具有相同数据类型的值的集合。例:

- 自然数、
- 整数
- 实数
- 总字符数度小于25的变长字符串集合
- {男，女}
- 0~100之间的正整数
- ...

# 关系

## 笛卡尔积

- 给定一组域$D_1$，$D_2$，…，$D_n$，允许其中某些域是相同的。$D_1$，$D_2$，...，$D_n$的笛卡尔积为：

$$D_1 \times D_2 \times \ldots \times D_n = \{（d_1，d_2，\ldots，d_n） \mid d_i \in D_i，i=1，2，\ldots，n\}$$

- 所有域的所有取值的一个组合
- 不能重复

# 关系

## 元组（Tuple）

- 笛卡尔积中每一个元素（d1，d2，…，dn）叫作一个n元组（n-tuple）或简称元组
- 例如：（1001，张三，北京），（1001，张三，上海） 等 都是元组

## 分量（Component）

- 笛卡尔积元素（d1，d2，…，dn）中的每一个值di 叫作一个分量
- 例如：“1001”，“张三”，“北京”等都是分量

# 关系

## 基数（Cardinal number）

- 若Di（i＝1，2，…，n）为有限集，其基数为mi（i＝1，2，…，n），则D1×D2×…×Dn的基数M为：

$$M = \prod_{i=1}^{n} m_i$$

## 笛卡尔积的表示方法

- 笛卡尔积可表示为一张二维表
- 表中的每行对应一个元组，表中的每列对应一个域

# 关系

例如，给出3个域：

- D1 = 顾客ID（Customer ID）={1001，1002，1003 }
- D2 = 顾客姓名（Customer Name）={张三，李四，王五}
- D3 = 城市（City）={北京，上海，广州}
- D1，D2，D3的笛卡尔积为

# 关系

- **D1×D2×D3 =**{

（1001，张三，北京），（1001，张三，上海），（1001，张三，广州），（1001，李四，北京），（1001，李四，上海），（1001，李四，广州），（1001，王五，北京），（1001，王五，上海），（1001，王五，广州），（1002，张三，北京），（1002，张三，上海），（1002，张三，广州），（1002，李四，北京），（1002，李四，上海），（1002，李四，广州），（1002，王五，北京），（1002，王五，上海），（1002，王五，广州），（1003，张三，北京），（1003，张三，上海），（1003，张三，广州），（1003，李四，北京），（1003，李四，上海），（1003，李四，广州），（1003，王五，北京），（1003，王五，上海），（1003，王五，广州）}。

- 其中，（1001，张三，北京），（1001，张三，上海）等都是元组，“1001”，“张三”，“北京”等都是元组的分量。
- 基数为3*3*3=27

# 关系

笛卡儿积示例

| 顾客ID | 顾客姓名 | 城市 |
| --- | --- | --- |
| 1001 | 张三 | 北京 |
| 1001 | 张三 | 上海 |
| 1001 | 张三 | 广州 |
| 1001 | 李四 | 北京 |
| 1001 | 李四 | 上海 |
| 1001 | 李四 | 广州 |
| 1001 | 王五 | 北京 |
| 1001 | 王五 | 上海 |
| 1001 | 王五 | 广州 |
| 1002 | 张三 | 北京 |
| …… | …… | …… |

假设学校有一千名老师，一千名研究生，一百个专业，一亿行！假设再多一些列。。。

# 关系

*D*1，*D*2，…，*D*n的笛卡尔积的某个子集才有实际含义

例：上述的笛卡尔积没有实际意义，我们取出有实际意义的元组来构造关系如下：

下表展示了顾客编号与顾客姓名和所在城市的关系。将该关系取名为Customers

Customers关系

| 顾客ID（CID） | 顾客姓名（CName） | 城市（City） |
|---|---|---|
| 1001 | 张三 | 北京 |
| 1002 | 李四 | 广州 |
| 1003 | 王五 | 上海 |

# 关系

## 关系

- 给定一组域D1，D2，…，Dn，允许其中某些域是相同的，D1×D2×…×Dn的笛卡儿积D1×D2×…×Dn的子集叫作在域D1，D2，…，Dn上的关系，表示为

  R（D1，D2，…，Dn）

- R：关系名
- n：关系的目或度（Degree）

# 关系

- 元组
  - 关系中的每个元素是关系中的元组，通常用*t*表示
- 单元关系与二元关系
  - 当*n*=1时，称该关系为单元关系（Unary relation）或一元关系
  - 当*n*=2时，称该关系为二元关系（Binary relation）

# 关系

- 关系的表示
  - 关系也是一个二维表，表的每行对应一个元组，表的每列对应一个域
- 属性
  - 关系中不同列可以对应相同的域
  - 为了加以区分，必须对每列起一个名字，称为属性（Attribute）
  - *n*目关系必有*n*个属性

# 关系

## 三类关系

- 基本关系（基本表或基表）--- 实际存在的表，是实际存储数据的逻辑表示
- 查询表--- 查询结果对应的表
- 视图表--- 由基本表或其他视图表导出的表，是虚表，不对应实际存储的数据

# 关系

## 基本关系的性质

① 列是同质的（Homogeneous）

② 不同的列可出自同一个域

- 其中的每一列称为一个属性
- 不同的属性要给予不同的属性名

③ 列的顺序无所谓,，列的次序可以任意交换

④ 任意两个元组的候选码不能相同

⑤ 行的顺序无所谓，行的次序可以任意交换

⑥ 分量必须取原子值，即每一个分量都必须是不可分的数据项。

# 基本关系的性质（续）

## ⑥ 分量必须取原子值

这是规范条件中最基本的一条

第6版 表2.3 非规范化关系

| SUPERVISOR | SPECIALITY | POSTGRADUATE | |
|---|---|---|---|
| | | PG1 | PG2 |
| 张清玫 | 计算机专业 | 李勇 | 刘晨 |
| 刘逸 | 信息专业 | 王敏 | |

小表

# 关系模型的数据结构及形式化定义

- 2.1 关系
- 2.2 关系模式
- 2.3 关系数据库
- 2.4 关系模型的存储结构

# 关系模式

- **关系模式是对关系的描述**
  - 元组集合的结构
    - 属性构成
    - 属性来自的域
    - 属性与域之间的映象关系
  - 完整性约束条件

# 关系模式

- 关系模式---对关系的描述
- 可以形式化地表示为：

$$R（U，D，DOM，F）$$

- $R$　　关系名
- $U$　　组成该关系的属性名集合
- $D$　　$U$中属性所来自的域
- DOM　　属性向域的映象集合
- $F$　　属性间数据的依赖关系的集合

# 关系模式

- 候选码(理论正确，但DB并不支持)
  - 关系模式中的某一个属性或一组属性的值能唯一地标识一个元组，而它的真子集不能唯一地标识一个元组
- 主码
  - 若一个关系有多个候选码，则选定其中一个为主码（primary key），或称主键。
- 主属性--候选码的诸属性
- 非主属性--不包含在任何候选码中的属性

# 关系模型的数据结构及形式化定义

- 2.1 关系
- 2.2 关系模式
- 2.3 关系数据库
- 2.4 关系模型的存储结构

# 关系数据库

- 关系数据库系统
  - 支持关系模型的数据库系统
  - 在关系模型中，实体以及实体间的联系都是用关系来表示的。
  - 在一个关系数据库中，某一时刻所有关系模式对应的关系的集合构成一个关系数据库。

**Customers关系**

| 顾客ID (CID) | 顾客姓名 (CName) | 城市 (City) |
|---|---|---|
| 1001 | 张三 | 北京 |
| 1002 | 李四 | 广州 |
| 1003 | 王五 | 上海 |

# 关系模型的数据结构及形式化定义

- 2.1 关系
- 2.2 关系模式
- 2.3 关系数据库
- 2.4 关系模型的存储结构

# 关系模型的存储结构

- 关系模型是关系数据的逻辑结构
  - 用关系数据定义语言描述。支持关系模型的关系数据库管理系统（RDBMS）将以一定的组织方式存储和管理数据，即设计和实现关系模型的存储结构，这是关系数据库管理系统的核心功能之一。
- 本书将在第5章介绍基于磁盘的数据库的组织与存储。

# 第二章 关系数据模型

- 2.1 关系模型的数据结构及形式化定义
- 2.2 关系代数
- 2.3 关系的完整性
- 2.4 小结

# 2.2 关系代数

- 关系代数是一种抽象的查询语言，它用对关系的运算来表达查询
- 关系代数
  - 运算对象是关系
  - 运算结果亦为关系
  - 关系代数的运算符有两类：集合运算符和专门的关系运算符
- 传统的集合运算是从关系的“水平”方向即行的角度进行
- 专门的关系运算不仅涉及行而且涉及列

# 2.2 关系代数

表 关系代数运算符

| 运 算 符 | | 含 义 |
|---|---|---|
| 集合运算符 | ∪ | 并 |
| | - | 差 |
| | ∩ | 交 |
| | × | 笛卡尔积 |
| 专门的关系运算符 | $\sigma$ | 选择 |
| | $\pi$ | 投影 |
| | ⋈ | 连接 |
| | ÷ | 除 |

# 关系代数

- 2.2.1 传统的集合运算
- 2.2.2 专门的关系运算

# 并（Union）

- $R$和$S$
  - 具有相同的目$n$（即两个关系都有$n$个属性）
  - 相应的属性取自同一个域
- $R \cup S$
  - 仍为$n$目关系，由属于$R$或属于$S$的元组组成

$$R \cup S = \{t | t_r \in R \vee t_s \in S\}$$

# 并（Union）

R

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a1 | b2 | c2 |
| a2 | b2 | c1 |

S

| A | B | C |
|---|---|---|
| a1 | b2 | c2 |
| a1 | b3 | c2 |
| a2 | b2 | c1 |

RUS

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a1 | b2 | c2 |
| a2 | b2 | c1 |
| a1 | b3 | c2 |

# 差（Difference）

- $R$和$S$
  - 具有相同的目$n$（即两个关系都有$n$个属性）
  - 相应的属性取自同一个域
- R - S
  - 仍为n目关系，由属于R而不属于S的所有元组组成

$$R - S = \{t | t_r \in R \ \wedge t_s \notin S\}$$

# 差（Difference）

R

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a1 | b2 | c2 |
| a2 | b2 | c1 |

S

| A | B | C |
|---|---|---|
| a1 | b2 | c2 |
| a1 | b3 | c2 |
| a2 | b2 | c1 |

R-S

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |

# 交（Intersection）

- $R$和$S$
  - 具有相同的目$n$（即两个关系都有$n$个属性）
  - 相应的属性取自同一个域
- R∩S
  - 仍为n目关系，由既属于R又属于S的元组组成

$$R \cap S = \{t | t_r \in R \ \wedge t_s \in S\}$$

# 交（Intersection）

R

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a1 | b2 | c2 |
| a2 | b2 | c1 |

S

| A | B | C |
|---|---|---|
| a1 | b2 | c2 |
| a1 | b3 | c2 |
| a2 | b2 | c1 |

R ∩ S

| A | B | C |
|---|---|---|
| a1 | b2 | c2 |
| a2 | b2 | c1 |

# 笛卡尔积（Cartesian Product）

严格地讲应该是广义的笛卡尔积（Extended Cartesian Product）

- R: $n$目关系，$k_1$个元组
- S: $m$目关系，$k_2$个元组

$R \times S$

- 列：（$n+m$）列元组的集合
  - 元组的前$n$列是关系$R$的一个元组
  - 后$m$列是关系$S$的一个元组
- 行：$k_1 \times k_2$个元组

$$R \times S = \{\widehat{t_r t_s} | t_r \in R \ \wedge t_s \in S\}$$

# 笛卡尔积（Cartesian Product）

R

| A | B | C |
|---|---|---|
| a1 | b1 | c1 |
| a1 | b2 | c2 |
| a2 | b2 | c1 |

S

| A | B | C |
|---|---|---|
| a1 | b2 | c2 |
| a1 | b3 | c2 |
| a2 | b2 | c1 |

R × S

| R.A | R.B | R.C | S.A | S.B | S.C |
|---|---|---|---|---|---|
| a1 | b1 | c1 | a1 | b2 | c2 |
| a1 | b1 | c1 | a1 | b3 | c2 |
| a1 | b1 | c1 | a2 | b2 | c1 |
| a1 | b2 | c2 | a1 | b2 | c2 |
| a1 | b2 | c2 | a1 | b3 | c2 |
| a1 | b2 | c2 | a2 | b2 | c1 |
| a2 | b2 | c1 | a1 | b2 | c2 |
| a2 | b2 | c1 | a1 | b3 | c2 |
| a2 | b2 | c1 | a2 | b2 | c1 |

# 关系代数

- 2.2.1 传统的集合运算
- 2.2.2 专门的关系运算

# 专门的关系运算

先引入几个记号

（1） R，t∈R，t[$A_i$]

设关系模式为$R(A_1, A_2, \ldots, A_n)$

它的一个关系设为$R$

$t \in R$表示$t$是$R$的一个元组

$t[A_i]$则表示元组$t$中相应于属性$A_i$的一个分量

# 专门的关系运算

（2） A，t[A]， $\overline{A}$

- 若$A=\{A_{i1}，A_{i2}，…，A_{ik}\}$，其中$A_{i1}$，$A_{i2}$，…，$A_{ik}$是$A_1$，$A_2$，…，$A_n$中的一部分，则A称为属性列或属性组。
- $t[A]=(t[A_{i1}]，t[A_{i2}]，…，t[A_{ik}])$表示元组t在属性列A上诸分量的集合。
- $\overline{A}$则表示$\{A_1，A_2，…，A_n\}$中去掉$\{A_{i1}，A_{i2}，…，A_{ik}\}$后剩余的属性组。

# 专门的关系运算（续）

(3) $\widehat{t_r t_s}$

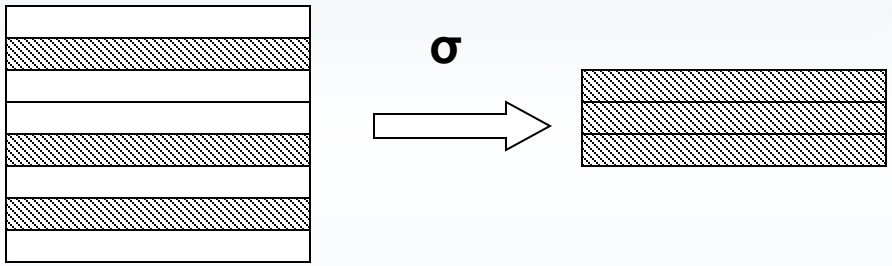
- $R$为$n$目关系，$S$为$m$目关系。
- $t_r \in R$，$t_s \in S$，$\widehat{t_r t_s}$称为元组的连接。
- $\widehat{t_r t_s}$是一个$n+m$列的元组，前$n$个分量为$R$中的一个$n$元组，后$m$个分量为$S$中的一个$m$元组。

# 选择

- 在关系 R 中选择满足给定条件的诸元组，记作

$$\sigma_F(R) = \{t | t \in R \wedge F(t) = \text{'True'}\}$$

- 其中 F 表示选择条件，它是一个逻辑表达式，取逻辑值“True”或“False”。 选择运算实际上是从关系R中选取使逻辑表达式F为“True”的元组。是从行的角度进行的运算。



# 选择

- 【例】查询种类（Category）为‘数码产品’的所有商品。

$$\sigma_{\text{Category}=\text{‘数码产品’}}(\text{Products})$$

- 结果如下表所示。

| 商品ID (PID) | 商品名称 (PName) | 价格 (Price) | 类别 (Category) | 供应商ID (SID) |
|---|---|---|---|---|
| P0001 | 智能手机 | 1999 | 数码产品 | S001 |
| P0002 | 老人专用手机 | 899 | 数码产品 | S001 |
| P0003 | 平板电脑 | 1688 | 数码产品 | S001 |

# 投影

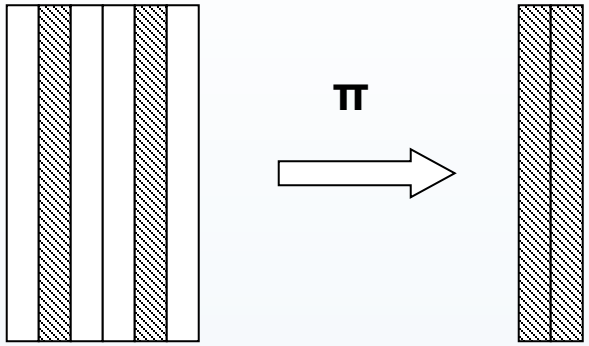
- 关系R上的投影是从R中选择若干属性列组成新的关系。 记作

$$\prod_{\mathbf{A}}(R)=\{t[\mathbf{A}]|t\in R\}$$

- 其中A是R的一组属性集合。

投影操作主要是从列的角度进行运算

投影之后不仅取消了原关系中的某些列，而且还可能取消某些元组（避免重复行）



# 投影

- [例]查询所有的顾客姓名和所在城市。

$$\prod_{CName,City}(Customers)$$

- 查询结果如下表所示。

查询所有的顾客姓名和所在城市

| 顾客姓名 (CName) | 城市 (City) |
|---|---|
| 张三 | 北京 |
| 李四 | 广州 |
| 王五 | 上海 |

# 连接（Join）

- **连接也称为θ连接**
- **连接运算的含义**

  从两个关系的笛卡尔积中选取属性间满足一定条件的元组

  $$R \underset{A\theta B}{\bowtie} S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B] \}$$

  - *A*和*B*：分别为*R*和*S*上度数相等且可比的属性组
  - **θ：比较运算符**

  - 连接运算从*R*和*S*的广义笛卡尔积*R*×*S*中选取*R*关系在*A*属性组上的值与*S*关系在*B*属性组上的值满足比较关系θ的元组

# 连接（续）

- **两类常用连接运算**
  - **等值连接**（equijoin）
    - θ为“ = ”的连接运算称为等值连接
    - 从关系*R*与*S*的广义笛卡尔积中选取*A*、*B*属性值相等的那些元组，即等值连接为：

$$R \underset{A=B}{\bowtie} S = \{ \overset{\frown}{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B] \}$$

# 连接（续）

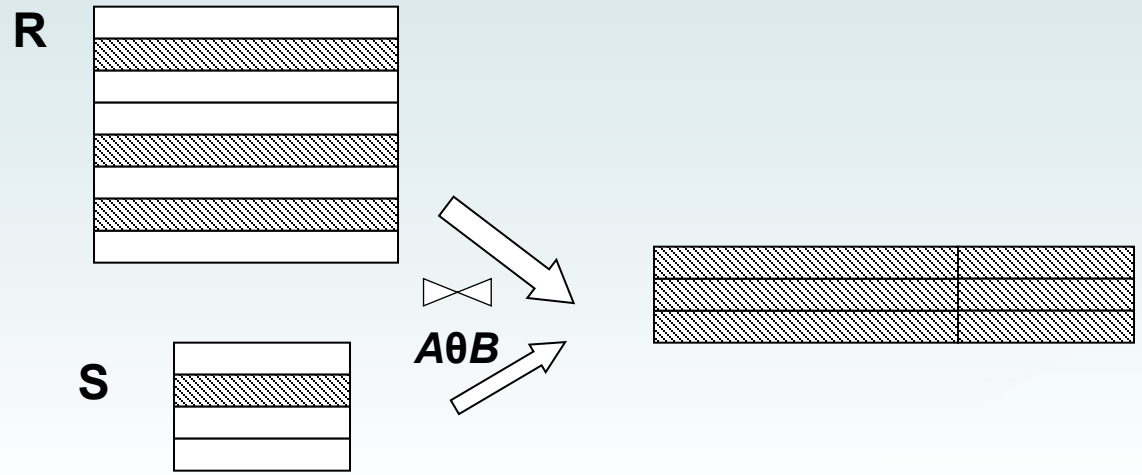
- 自然连接（Natural join）
  - 自然连接是一种特殊的等值连接
    - 两个关系中进行比较的分量必须是相同的属性组
    - 在结果中把重复的属性列去掉
  - 自然连接的含义

    R和S具有相同的属性组B

    $R \bowtie S = \{ \widehat{t_r t_s}\,[U-B] \mid t_r \in R \wedge t_s \in S \wedge t_r[B] = t_s[B] \}$

# 连接（续）

- 一般的连接操作是从行的角度进行运算。




自然连接还需要取消重复列，所以是同时从行和列的角度进行运算。

# 连接

- 对于顾客关系（Customers,下面简称C）和订单关系（Orders，下面简称O）

Customers关系

| 顾客ID (CID) | 顾客姓名 (CName) | 城市 (City) |
|---|---|---|
| 1001 | 张三 | 北京 |
| 1002 | 李四 | 广州 |
| 1003 | 王五 | 上海 |

Orders关系

| 订单编号 (OID) | 顾客ID (CID) | 创建时间 (CreateTime) |
|---|---|---|
| O001 | 1001 | 2023-1-1 18:00:10 |
| O002 | 1002 | 2023-1-1 18:20:01 |
| O003 | 1002 | 2023-1-5 09:10:20 |

# 连接

| C.CID | CName | City | OID | O.CID | CreateTime |
|---|---|---|---|---|---|
| 1002 | 李四 | 广州 | O001 | 1001 | 2023-1-1 18:00:10 |
| 1003 | 王五 | 上海 | O002 | 1002 | 2023-1-1 18:20:01 |
| 1003 | 王五 | 上海 | O003 | 1002 | 2023-1-5 09:10:20 |

注：教材此例缺乏实际含义

# 连接

| C.CID | CName | City | OID | O.CID | CreateTime |
|---|---|---|---|---|---|
| 1001 | 张三 | 北京 | O001 | 1001 | 2023-1-1 18:00:10 |
| 1002 | 李四 | 广州 | O002 | 1002 | 2023-1-1 18:20:01 |
| 1002 | 李四 | 广州 | O003 | 1002 | 2023-1-5 09:10:20 |
| 1003 | 王五 | 上海 | O004 | 1003 | 2023-2-5 19:20:10 |

注：得到客户与订单信息，比如前端呈现并不表示表结构设计

# 连接

| CID | CName | City | OID | CreateTime |
|---|---|---|---|---|
| 1001 | 张三 | 北京 | O001 | 2023-1-1 18:00:10 |
| 1002 | 李四 | 广州 | O002 | 2023-1-1 18:20:01 |
| 1002 | 李四 | 广州 | O003 | 2023-1-5 09:10:20 |
| 1003 | 王五 | 上海 | O004 | 2023-2-5 19:20:10 |

Q:与上图有什么区别？

# 连接（续）

- 第6版[例2.8]关系*R*和关系*S* 如下所示：

R

| A | B | C |
|---|---|---|
| a1 | b1 | 5 |
| a1 | b2 | 6 |
| a2 | b3 | 8 |
| a2 | b4 | 12 |

S

| B | E |
|---|---|
| b1 | 3 |
| b2 | 7 |
| b3 | 10 |
| b3 | 2 |
| b5 | 2 |

R

| A | B | C |
|---|---|---|
| a1 | b1 | 5 |
| a1 | b2 | 6 |
| a2 | b3 | 8 |
| a2 | b4 | 12 |

S

| B | E |
|---|---|
| b1 | 3 |
| b2 | 7 |
| b3 | 10 |
| b3 | 2 |
| b5 | 2 |

一般连接 $R \bowtie_{C<E} S$的结果如下:

| A | R.B | C | S.B | E |
|---|---|---|---|---|
| a1 | b1 | 5 | b2 | 7 |
| a1 | b1 | 5 | b3 | 10 |
| a1 | b2 | 6 | b2 | 7 |
| a1 | b2 | 6 | b3 | 10 |
| a2 | b3 | 8 | b3 | 10 |

R

| A | B | C |
|---|---|---|
| a1 | b1 | 5 |
| a1 | b2 | 6 |
| a2 | b3 | 8 |
| a2 | b4 | 12 |

S

| B | E |
|---|---|
| b1 | 3 |
| b2 | 7 |
| b3 | 10 |
| b3 | 2 |
| b5 | 2 |

等值连接 $R \underset{R.B=S.B}{\bowtie} S$ 的结果如下:

| A | R.B | C | S.B | E |
|---|---|---|---|---|
| a1 | b1 | 5 | b1 | 3 |
| a1 | b2 | 6 | b2 | 7 |
| a2 | b3 | 8 | b3 | 10 |
| a2 | b3 | 8 | b3 | 2 |

# 连接（续）

自然连接 $R \bowtie S$的结果如下：

| A | B | C | E |
|---|---|---|---|
| a1 | b1 | 5 | 3 |
| a1 | b2 | 6 | 7 |
| a2 | b3 | 8 | 10 |
| a2 | b3 | 8 | 2 |

# 连接（续）

- **悬浮元组（Dangling tuple）**
  - 两个关系*R*和*S*在做自然连接时，关系*R*中某些元组有可能在*S*中不存在公共属性上值相等的元组，从而造成*R*中这些元组在操作时被舍弃了，这些被舍弃的元组称为悬浮元组。

# 连接（续）

- **外连接（Outer Join）**
  - 如果把悬浮元组也保存在结果关系中，而在其他属性上填空值(Null)，就叫做外连接
  - 左外连接(LEFT OUTER JOIN或LEFT JOIN)
    - 只保留左边关系*R*中的悬浮元组
  - 右外连接(RIGHT OUTER JOIN或RIGHT JOIN)
    - 只保留右边关系*S*中的悬浮元组

# 连接（续）

下图是例2.8中关系*R*和关系*S*的外连接

| A | B | C | E |
|---|---|---|---|
| a1 | b1 | 5 | 3 |
| a1 | b2 | 6 | 7 |
| a2 | b3 | 8 | 10 |
| a2 | b3 | 8 | 2 |
| a2 | b4 | 12 | NULL |
| NULL | b5 | NULL | 2 |

# 连接（续）

图(b)是例2.8中关系*R*和关系*S*的左外连接,图(c)是右外连接

| A | B | C | E |
|---|---|---|---|
| a1 | b1 | 5 | 3 |
| a1 | b2 | 6 | 7 |
| a2 | b3 | 8 | 10 |
| a2 | b3 | 8 | 2 |
| a2 | b4 | 12 | NULL |

图(b)

| A | B | C | E |
|---|---|---|---|
| a1 | b1 | 5 | 3 |
| a1 | b2 | 6 | 7 |
| a2 | b3 | 8 | 10 |
| a2 | b3 | 8 | 2 |
| NULL | b5 | NULL | 2 |

图(c)

# Student表

**学生-课程数据库:**

**学生关系Student表、课程关系Course表和选修关系SC表**

| 学号<br>Sno | 姓名<br>Sname | 性别<br>Ssex | 年龄<br>Sage | 所在系<br>Sdept |
|---|---|---|---|---|
| 201215121 | 李勇 | 男 | 20 | CS |
| 201215122 | 刘晨 | 女 | 19 | CS |
| 201215123 | 王敏 | 女 | 18 | MA |
| 201215125 | 张立 | 男 | 19 | IS |

Q：某学号在本表最多有几条记录？

# Course表

| 课程号<br>Cno | 课程名<br>Cname | 先行课<br>Cpno | 学分<br>Ccredit |
|---|---|---|---|
| 1 | 数据库 | 5 | 4 |
| 2 | 高等数学 | | 2 |
| 3 | 信息系统 | 1 | 4 |
| 4 | 操作系统 | 6 | 3 |
| 5 | 数据结构 | | 4 |
| 6 | 数据处理 | | 2 |
| 7 | Java语言 | 5 | 4 |

Q：某cno在本表最多有几条记录？

# SC表

| 学 号<br>Sno | 课程号<br>Cno | 成绩<br>Grade |
|---|---|---|
| 201215121 | 1 | 92 |
| 201215121 | 2 | 85 |
| 201215121 | 3 | 88 |
| 201215122 | 2 | 90 |
| 201215122 | 3 | 80 |

Q：某组(sno+cno) 最多有几条记录？会出现什么问题？

一次挂科后，无法毕业，后续E-R图详解。

Q：为何本表没有姓名和课程名？但前端可以呈现？

RDB的关键特征，后续E-R图详解。可以多表查询。

# 综合举例（续）

补充例题：查询选修了2号课程的学生的学号。

$\pi_{Sno}(\sigma_{Cno='2'}(SC))=\{201215121,201215122\}$

补充例题：查询至少选修了一门其直接先行课为5号课程的学生姓名

$\pi_{Sname}(\sigma_{Cpno='5'}(Course \bowtie SC \bowtie \pi_{Sno,Sname}(Student))$

或

$\pi_{Sname}(\pi_{Sno}(\sigma_{Cpno='5'}(Course) \bowtie SC) \bowtie \pi_{Sno,Sname}(Student))$

# 第二章 关系数据模型

- 2.1 关系模型的数据结构及形式化定义
- 2.2 关系代数
- 2.3 关系的完整性
- 2.4 小结

# 2.3 关系的完整性

- 实体完整性和参照完整性
  - 关系模型必须满足的完整性约束条件称为关系的两个不变性，应该由关系系统自动支持
- 用户定义的完整性
  - 应用领域需要遵循的约束条件，体现了具体领域中的语义约束

# 2.3 关系的完整性

- 2.3.1 实体完整性
- 2.3.2 参照完整性
- 2.3.3 用户定义的完整性

# 2.3.1 实体完整性

- 实体完整性目的是保证关系数据库中每个元组是可区分的，并且是**唯一**的
- 实体完整性约束
  - 若属性*A*是基本关系*R*的主属性，则属性*A*不能取空值
  - 空值就是“不知道”或“不存在”或“无意义”的值
- 例：
  - Customers关系的主码为CID，则CID不可为空。
  - 注意，若关系主码包含多个属性，则每个属性均不能为空。例如，OrderItems关系的主码为（OID，PID），则两个都不能取空值。

# 2.3.1 实体完整性

- 实体完整性规则的说明

（1）实体完整性规则是针对基本关系而言的。一个基本表通常对应现实世界的一个实体集。例如，“Customers”关系对应顾客的集合。

（2）现实世界中的实体是可区分的，即它们具有某种唯一性标识。例如，每个顾客都是独立的个体，是不一样的。

（3）关系模型中以主码作为唯一性标识。

（4）**主码中的属性即主属性不能取空值**。

主属性假如取空值，就说明存在某个不可标识的实体，即存在不可区分的实体，这与第（2）点相矛盾，因此这个规则称为实体完整性

# 2.3 关系的完整性

- 2.3.1 实体完整性
- 2.3.2 参照完整性
- 2.3.3 用户定义的完整性

# 2.3.2 参照完整性

1. 关系间的引用
2. 外码
3. 参照完整性规则

# 2.3.2 参照完整性

- 现实世界中的实体之间往往存在某种联系，在关系模型中实体及实体间的联系都是用关系来描述的，这样就自然存在着关系与关系间的引用。
- 例：**“供应商”和“商品”**可以用下面的关系模式来表示。其中下横线表示该属性为主码。
  - 供应商（供应商ID，供应商名称，城市）
  - 商品（商品ID，商品名称，价格，类别，供应商ID）
  - 商品关系引用了供应商关系的主码“供应商ID”。在商品关系中出现的供应商ID必须是真实存在的供应商的ID，也就是说，商品关系中“供应商ID”的取值要参照供应商关系中“供应商ID”的取值。

# 2.3.2 参照完整性

- 设*F*是基本关系*R*的一个或一组属性，但不是关系*R*的码。如果F与基本关系*S*的主码$K_s$相对应，则称F是*R*的**外码**
- 基本关系*R*称为参照关系（Referencing Relation）
- 基本关系*S*称为被参照关系（Referenced Relation）或目标关系（Target Relation）
- 在商品关系的例子中，商品关系中的“供应商ID”是外码，参照了供应商关系的主码“供应商ID”。商品关系是参照关系，供应商关系是被参照关系。

# 2.3.2 参照完整性

- 参照完整性规则
  - 若属性（或属性组）$F$是基本关系$R$的外码它与基本关系$S$的主码$K_s$相对应（基本关系$R$和$S$不一定是不同的关系），则对于$R$中每个元组在$F$上的值必须为：
    - 或者取空值（$F$的每个属性值均为空值）
    - 或者等于$S$中某个元组的主码值
- 例如，在商品关系的例子中，商品关系中的“供应商ID”或者为空值，或者等于供应商关系中某个元组的“供应商ID”。

# 关系间的引用（续）

**补充例题：学生实体及其内部的一对多联系**

学生（学号，姓名，性别，专业号，年龄，班长）

| 学号 | 姓名 | 性别 | 专业号 | 年龄 | 班长 |
|---|---|---|---|---|---|
| 801 | 张三 | 女 | 01 | 19 | 802 |
| 802 | 李四 | 男 | 01 | 20 | |
| 803 | 王五 | 男 | 01 | 20 | 802 |
| 804 | 赵六 | 女 | 02 | 20 | 805 |
| 805 | 钱七 | 男 | 02 | 19 | |

- “学号”是主码，“班长”是外码，它引用了本关系的“学号”
- “班长” 必须是确实存在的学生的学号

# 2.3 关系的完整性

- 2.3.1 实体完整性
- 2.3.2 参照完整性
- 2.3.3 用户定义的完整性

# 2.3.3 用户定义的完整性

- 针对某一具体关系数据库的约束条件，反映某一具体应用所涉及的数据必须满足的语义要求
- 关系模型应提供定义和检验这类完整性的机制，以便用统一的系统的方法处理它们，而不需由应用程序承担这一功能
- 【例如】
  - 要求供应商名称不能重复
  - 要求商品价格必须大于0
  - 要求订单项关系中的折扣必须是0~1之间的小数

# 第二章 关系数据模型

- 2.1 关系模型的数据结构及形式化定义
- 2.2 关系代数
- 2.3 关系的完整性
- 2.4 小结

# 2.4 小结

- 主要介绍
  - 关系模型的数据结构和形式化定义
  - 关系代数
  - 关系的完整性
- 难点：关系代数，特别是里面的连接运算

# 小结

- **关系代数运算**
  - 关系代数运算
    - 并、差、交、笛卡尔积、投影、选择、连接、除(与SQL实现不同，不讲)
  - 基本运算
    - 并、差、笛卡尔积、投影、选择
  - 交、连接
    - 可以用5种基本运算来表达
    - 引进它们并不增加语言的能力，但可以简化表达

# 课后习题

- 第二章课后习题：1，3，第6版第二章6 (1)-(4)(只用关系代数）
  - 简要作答
- 提交链接：微信群课后分享